

PROJET — RAPPORT

Methodes de gradient proximal

ISTA & FISTA pour l'optimisation convexe non lisse

Convex optimization 2025–2026

N'Diaye BALL, Melvyn POMMIER, Romain REN, Jacq VENGADESAN

Ce projet a été réalisé avec l'aide d'un assistant conversationnel fondé sur l'IA, utilisé pour la structuration du document et du notebook, l'aide à la rédaction et la mise en forme des expressions mathématiques (L^AT_EX). Les choix méthodologiques, l'analyse des algorithmes, les preuves et les expériences numériques relèvent de notre travail.

June 7, 2026

Contents

1	Introduction et motivation	3
1.1	Du cours a une question laissee ouverte	3
1.2	La technique etudiee : les methodes de gradient proximal	3
1.3	Fil conducteur et application illustrative	3
1.4	Organisation du rapport	3
2	Rappels et cadre theorique	4
2.1	Probleme d'optimisation	4
2.2	Convexite	4
2.3	La propriete fondamentale, et ses limites	4
2.4	Programmation lineaire et quadratique	5
3	Le probleme : minimisation convexe composite	5
3.1	Un probleme inverse sous-determine	5
3.2	Formulation : le LASSO	5
3.3	Le probleme est convexe	6
3.4	Pourquoi les methodes du cours echouent	6
4	Outils theoriques : sous-differentiel et operateur proximal	6
4.1	Le sous-differentiel	7
4.2	Condition d'optimalite	7
4.3	L'operateur proximal	7
4.4	Derivation du seuillage doux	8
4.5	Non-expansivite	9
5	ISTA : l'algorithme de gradient proximal	9
5.1	Idee : separer la partie lisse et la partie non lisse	9
5.2	Construction par majoration-minimisation	9
5.3	L'algorithme	10
5.4	Convergence en $O(1/k)$	10
6	FISTA : acceleration par momentum	11
6.1	Du gradient proximal au momentum de Nesterov	11
6.2	L'algorithme	11
6.3	Convergence en $O(1/k^2)$	12
6.4	Non-monotonie et variantes a restart	12
7	Etat de l'art : situer ISTA et FISTA	13
7.1	Le cadre commun : minimiser une somme $f + g$	13
7.2	Descente par coordonnees	13
7.3	ADMM (Alternating Direction Method of Multipliers)	14
7.4	Newton proximal	15
7.5	Synthese comparative	15
8	Expériences numériques	16
8.1	Effet de λ : parcimonie contre fidelite	16
8.2	Effet du pas t : la frontiere de stabilite	17

9 Application LASSO/FISTA : audio de trompette	18
9.1 Formuler le débruitage comme un LASSO	18
9.2 Mesure de qualité : le SDR	18
9.3 Résultats	19
10 Conclusion	19
Références	20

1 Introduction et motivation

1.1 Du cours a une question lisee ouverte

Le cours d'optimisation convexe etablit un resultat fondamental : pour une fonction convexe sur un ensemble convexe, *tout minimum local est un minimum global*. Cette propriete — que nous appellerons la « propriete-or » — est ce qui rend les problemes convexes « faciles » a optimiser : il n'existe pas de creux sous-optimal dans lequel un algorithme pourrait rester piege.

Le cours a ensuite presente deux familles de methodes de resolution :

- pour un objectif **lisse** (differentiable), les methodes fondees sur le gradient (descente de gradient, methode de Newton) ;
- pour un objectif **lineaire**, l'algorithme du simplexe.

Ces deux familles couvrent une large part des problemes, mais elles partagent une hypothese implicite forte : la descente de gradient et la methode de Newton exigent que la fonction objectif soit *differentiable*, tandis que le simplexe ne traite que le cas *lineaire*. Une question reste donc ouverte, et c'est precisement celle qu'explore ce projet :

*Comment resoudre un probleme convexe lorsque la fonction objectif est convexe mais **ni lisse**, **ni lineaire** ?*

Ce cas de figure est loin d'etre marginal : il apparait des que l'on ajoute a un objectif lisse un terme de regularisation non differentiable, situation omnipresente en traitement du signal, en statistique et en apprentissage automatique.

1.2 La technique etudiee : les methodes de gradient proximal

Pour repondre a cette question, nous etudions une famille de methodes de premier ordre adaptees au cas non lisse : les **methodes de gradient proximal**, et en particulier l'algorithme **ISTA** (*Iterative Shrinkage-Thresholding Algorithm*) et son acceleration **FISTA** (*Fast ISTA*). Conformement au sujet B, le cœur de ce rapport est l'*etude de cette technique de resolution* : sa construction, ses garanties de convergence, et sa comparaison avec les alternatives existantes.

Il importe de souligner d'emblee qu'ISTA et FISTA *ne sont pas des algorithmes d'apprentissage automatique* : ce sont des algorithmes d'optimisation convexe, au meme titre que le gradient conjuge ou les methodes de point interieur cites dans l'enonce du sujet B. Le fait qu'ils soient frequemment employes en apprentissage ne change rien a leur nature : ils resolvent un probleme d'optimisation, point.

1.3 Fil conducteur et application illustrative

Pour ancrer l'etude dans un probleme concret, nous utilisons comme fil rouge le probleme du **LASSO** (regression lineaire avec penalisation L_1), qui est l'instance canonique d'un objectif convexe composite « lisse + non lisse ». En fin de rapport, nous illustrons l'interet pratique de la methode sur une **application a la separation de sources audio** (section 9) ; cette application reste toutefois une *illustration* et non le sujet : elle rend le probleme tangible et fournit une evaluation quantitative, mais l'analyse demeure centree sur l'optimisation.

1.4 Organisation du rapport

Le rapport s'organise en trois temps. La premiere partie pose le cadre : rappels du cours mobilises (section 2) puis formulation precise du probleme composite et preuve de sa convexite (section 3). La deuxieme partie constitue le cœur theorique : les outils de l'optimisation non lisse, dont l'operateur proximal (section 4), puis les algorithmes ISTA (section 5) et FISTA (section 6) avec

leurs preuves de convergence. La troisième partie met la méthode en perspective : état de l'art comparatif (section 7), application audio (section 9) et expériences numériques (section 9.3). La section 10 conclut.

2 Rappels et cadre théorique

Cette section rassemble les notions du cours mobilisées dans la suite. Elle ne vise pas l'exhaustivité mais fixe le vocabulaire, les notations et les propriétés sur lesquelles reposent les sections suivantes.

2.1 Problème d'optimisation

Un problème d'optimisation consiste à minimiser (ou maximiser) une fonction objectif $f : A \rightarrow \mathbb{R}$ sur un ensemble admissible A :

$$\min_{x \in A} f(x).$$

Un élément $x \in A$ est une *solution admissible* ; une solution qui réalise le minimum est une *solution optimale*, notée x^* . Lorsque l'ensemble admissible est décrit par des contraintes d'inégalité, on écrit

$$\min_{x \in \mathbb{R}^n} f(x) \quad \text{sous contraintes} \quad c_i(x) \leq b_i, \quad i = 1, \dots, m.$$

2.2 Convexité

[11]

Définition 1 (Ensemble convexe). Un ensemble $A \subseteq \mathbb{R}^n$ est *convexe* si, pour tout couple de points de A , le segment qui les joint est entièrement contenu dans A :

$$\forall x, y \in A, \forall \lambda \in [0, 1] : \quad \lambda x + (1 - \lambda)y \in A.$$

Définition 2 (Fonction convexe). Une fonction $f : \mathbb{R}^n \rightarrow \mathbb{R}$ est *convexe* si

$$\forall x, y \in \mathbb{R}^n, \forall \lambda \in [0, 1] : \quad f(\lambda x + (1 - \lambda)y) \leq \lambda f(x) + (1 - \lambda)f(y).$$

Géométriquement, tout segment reliant deux points du graphe se situe au-dessus (ou sur) le graphe. Lorsque l'inégalité est *stricte* pour $x \neq y$ et $\lambda \in]0, 1[$, la fonction est dite *strictement convexe*.

Deux propriétés de stabilité, établies en travaux dirigés, seront utilisées pour montrer que notre problème est convexe :

Proposition 1 (Stabilité de la convexité).

1. Un demi-espace $\{x \in \mathbb{R}^n : a^\top x \leq b\}$ est un ensemble convexe ; une intersection d'ensembles convexes est convexe.
2. Une somme de fonctions convexes est convexe ; le produit d'une fonction convexe par un scalaire positif est convexe.

2.3 La propriété fondamentale, et ses limites

Théorème 1 (Propriété-or). *Tout minimum local d'une fonction convexe est un minimum global.*

C'est cette propriété qui justifie l'intérêt des problèmes convexes : un algorithme qui converge vers un minimum local converge en réalité vers l'optimum global. Une précision s'impose toutefois, car elle conditionne la lecture de nos résultats expérimentaux.

Remarque 1 (Convexite et unicite). La convexite garantit l'unicite de la *valeur optimale* F^* , mais **pas** celle du *minimiseur* x^* . L'ensemble des minimiseurs peut etre reduit a un point, mais aussi former tout un segment — le cours en donne d'ailleurs un exemple en programmation lineaire, ou l'ensemble des solutions optimales est un segment entier. L'unicite du minimiseur requiert une hypothese plus forte, la *stricte* convexite. Comme nous le verrons (section 3), le LASSO n'est pas strictement convexe en general ; on ne pourra donc affirmer que la convergence se fait vers une *meme valeur* F^* , et non necessairement vers un *meme point* x^* .

2.4 Programmation lineaire et quadratique

Le cours a etudie en detail la *programmation lineaire*, cas particulier ou objectif et contraintes sont lineaires, resolue par l'algorithme du simplexe. Notre probleme generalise ce cadre dans une direction differente : l'objectif y sera *quadratique* (donc convexe mais non lineaire) et augmente d'un terme de regularisation *non differentiable*. Ni le simplexe ni la descente de gradient classique ne s'y appliquent directement, ce qui motive l'introduction des outils de la section 4.

3 Le probleme : minimisation convexe composite

3.1 Un probleme inverse sous-determine

Placons-nous dans une situation typique du traitement du signal. On observe un vecteur de mesures $y \in \mathbb{R}^n$, relie a un signal inconnu $x \in \mathbb{R}^p$ par une matrice connue $A \in \mathbb{R}^{n \times p}$:

$$y = Ax + \varepsilon,$$

ou ε represente un bruit. Le point determinant est que l'on dispose de *moins de mesures que d'inconnues* ($p > n$) : le systeme est *sous-determine* et admet une infinite de solutions compatibles avec y . Sans information supplementaire, le probleme est mal pose.

L'information que l'on ajoute est une hypothese de **parcimonie** (*sparsity*) : le vrai signal x ne possede que peu de composantes non nulles. Cette hypothese est realiste — de nombreux signaux naturels admettent une representation creuse dans une base adaptee — et suffit a rendre le probleme bien pose.

3.2 Formulation : le LASSO

Pour retrouver x , on minimise un objectif en deux termes, l'un mesurant la fidelite aux donnees, l'autre favorisant la parcimonie :

$$\min_{x \in \mathbb{R}^p} F(x) = \underbrace{\frac{1}{2} \|Ax - y\|_2^2}_{f(x) \text{ (lisse)}} + \lambda \underbrace{\|x\|_1}_{g(x) \text{ (non lisse)}} \quad (1)$$

ou $\|x\|_1 = \sum_{i=1}^p |x_i|$ et $\lambda > 0$ regle le compromis entre les deux termes. Ce probleme est connu sous le nom de **LASSO** [1] (*Least Absolute Shrinkage and Selection Operator*). On notera dans toute la suite

$$f(x) = \frac{1}{2} \|Ax - y\|_2^2, \quad g(x) = \lambda \|x\|_1, \quad F = f + g.$$

Le terme f est differentiable, de gradient

$$\nabla f(x) = A^\top (Ax - y), \quad (2)$$

qui est *lipschitzien* de constante $L = \|A^\top A\|_2$ (la plus grande valeur propre de $A^\top A$). Cette constante jouera un role central dans le choix du pas des algorithmes (sections 5 et 6).

3.3 Le probleme est convexe

Verifions que (1) entre bien dans le cadre du cours.

Proposition 2. *La fonction objectif $F = f + g$ est convexe sur $\mathbb{R}^p$, et l'ensemble admissible $\mathbb{R}^p$ est convexe. Le probleme (1) est donc un probleme d'optimisation convexe.*

Proof. On procede terme a terme.

1. *f est convexe.* La fonction $f(x) = \frac{1}{2} \|Ax - y\|_2^2$ est quadratique, de matrice hessienne $\nabla^2 f(x) = A^\top A$. Pour tout $u \in \mathbb{R}^p$, $u^\top A^\top A u = \|Au\|_2^2 \geq 0$, donc $A^\top A$ est semi-definie positive : f est convexe.
2. *g est convexe.* La norme $\|\cdot\|_1$ est une norme, donc convexe (inegalite triangulaire et homogeneite) ; multipliee par $\lambda > 0$, elle reste convexe (proposition 1).
3. *F est convexe.* Comme somme de deux fonctions convexes, F est convexe (proposition 1).
4. *L'ensemble admissible.* Il s'agit de $\mathbb{R}^p$ tout entier, qui est trivialement convexe.

La propriete-or (theoreme 1) s'applique donc : tout minimum local de (1) est global. □

Remarque 2 (Pas de stricte convexite en general). Conformement a la remarque 1, F n'est pas strictement convexe des que $p > n$. En effet, f n'est strictement convexe que si $A^\top A$ est *definie* positive, c'est-a-dire si A est injective ; or, avec plus de colonnes que de lignes ($p > n$), A possede un noyau non trivial. Quant a $g = \lambda \|\cdot\|_1$, elle est convexe mais pas strictement. On ne peut donc pas garantir l'unicite du minimiseur x^* : les algorithmes des sections suivantes convergeront vers une *memme valeur optimale* F^* , sans que le point atteint soit necessairement unique.

3.4 Pourquoi les methodes du cours echouent

Le terme $g(x) = \lambda \|x\|_1$ fait intervenir des valeurs absolues $|x_i|$. Or la fonction $t \mapsto |t|$ presente un *point anguleux* en 0 : sa derivee vaut -1 a gauche et $+1$ a droite, et **n'existe pas** en 0. Cette simple observation a des consequences directes :

- la **descente de gradient** requiert ∇F , qui n'est pas defini la ou des composantes de x s'annulent — c'est-a-dire precisement la ou se trouve la solution recherchee, puisqu'elle est creuse ;
- la **methode de Newton** exige en outre la hessienne de F , encore moins disponible ;
- le **simplexe** ne traite que les objectifs lineaires, alors que le notre est quadratique.

Il nous faut donc un outil capable de gerer ce point anguleux sans recourir a la differentiability. Cet outil est l'*operateur proximal*, introduit a la section suivante.

4 Outils theoriques : sous-differentiel et operateur proximal

Le terme de parcimonie $g(x) = \lambda \|x\|_1$ n'est pas derivable, ce qui interdit l'usage direct du gradient. Cette section construit les deux outils qui lèvent l'obstacle : le **sous-différentiel**, qui généralise le gradient aux fonctions convexes non lisses, et l'**opérateur proximal**, qui généralise le pas de gradient. On en déduit la formule du **seuillage doux**, cœur des algorithmes ISTA et FISTA.

4.1 Le sous-différentiel

Pour une fonction convexe dérivable, un minimum x^* vérifie $\nabla F(x^*) = 0$. Lorsque la fonction n'est pas dérivable, on remplace le gradient par le **sous-différentiel**. Pour une fonction convexe h , le sous-différentiel en x est l'ensemble des pentes v dont la droite associée reste sous le graphe de h :

$$\partial h(x) = \{ v : h(u) \geq h(x) + v^\top(u - x), \forall u \}. \quad (3)$$

Chaque $v \in \partial h(x)$ est appelé un *sous-gradient*. La définition généralise bien le gradient : lorsque h est dérivable en x , le sous-différentiel se réduit à un singleton, $\partial h(x) = \{\nabla h(x)\}$. Le sous-différentiel n'est donc pas un objet nouveau et déconnecté, mais une *extension* naturelle du gradient aux points anguleux.

Le cas qui nous intéresse est la valeur absolue, dont le sous-différentiel vaut

$$\partial |s| = \begin{cases} \{1\}, & s > 0, \\ [-1, 1], & s = 0, \\ \{-1\}, & s < 0. \end{cases} \quad (4)$$

En tout point $s \neq 0$, la fonction est dérivable et la pente est unique (± 1). Au point $s = 0$, en revanche, il n'existe pas de pente unique : **toutes les pentes de l'intervalle** $[-1, 1]$ sont compatibles avec le coin de la fonction. C'est précisément cette multiplicité au point non dérivable qui manquait à la descente de gradient classique, et que le sous-différentiel capture.

4.2 Condition d'optimalité

Le sous-différentiel fournit une condition d'optimalité valable même pour les fonctions non lisses. Pour une fonction convexe F , le point x^* est un minimiseur global si et seulement si le vecteur nul est un sous-gradient en ce point :

$$x^* \text{ minimise } F \iff 0 \in \partial F(x^*). \quad (5)$$

C'est la généralisation directe de la condition $\nabla F(x^*) = 0$ du cas dérivable. Appliquée au problème LASSO $F(x) = \frac{1}{2} \|Ax - y\|_2^2 + \lambda \|x\|_1$, dont la partie lisse a pour gradient $\nabla f(x) = A^\top(Ax - y)$, elle donne

$$0 \in A^\top(Ax^* - y) + \lambda \partial \|x^*\|_1. \quad (6)$$

On retrouve l'intuition attendue : au minimum, la force qui pousse à améliorer l'ajustement aux données (le terme de gradient) est exactement compensée par la force qui pousse les coefficients vers zéro (le sous-gradient de la pénalité).

4.3 L'opérateur proximal

[9]

La condition d'optimalité (5) caractérise la solution, mais ne dit pas comment l'atteindre. L'idée des méthodes proximales est de **séparer** le traitement des deux morceaux de F : un pas de gradient ordinaire sur la partie lisse f , et un *pas proximal* sur la partie non lisse g . L'opérateur proximal de g avec un pas $t > 0$ est défini par

$$\text{prox}_{tg}(z) = \arg \min_u \left(g(u) + \frac{1}{2t} \|u - z\|_2^2 \right). \quad (7)$$

L'interprétation est géométrique : on cherche un point u qui *reste proche* de z (grâce au terme quadratique $\frac{1}{2t} \|u - z\|_2^2$) tout en *réduisant* $g(u)$. Le paramètre t règle l'équilibre entre ces deux exigences. Le terme quadratique étant fortement convexe, le minimiseur de (7) est unique : l'opérateur proximal est une fonction bien définie. C'est une façon de faire un « pas » sur une fonction même là où elle n'est pas dérivable.

4.4 Derivation du seuillage doux

[10]

On calcule maintenant explicitement l'opérateur proximal de la pénalité $g(u) = \lambda \|u\|_1$, qui est le cœur de toute la méthode. Posons $\alpha = t\lambda$. Comme multiplier l'objectif par la constante $t > 0$ ne change pas son minimiseur,

$$\text{prox}_{t\lambda\|\cdot\|_1}(z) = \arg \min_u \alpha \|u\|_1 + \frac{1}{2} \|u - z\|_2^2. \quad (8)$$

Séparation coordonnée par coordonnée. L'objectif de (8) se décompose en une somme de termes indépendants, un par composante :

$$\alpha \|u\|_1 + \frac{1}{2} \|u - z\|_2^2 = \sum_i \left(\alpha |u_i| + \frac{1}{2} (u_i - z_i)^2 \right). \quad (9)$$

Minimiser la somme revient donc à minimiser *séparément* chaque terme. Le problème vectoriel se ramène à un problème **scalaire** identique sur chaque coordonnée :

$$\min_u \alpha |u| + \frac{1}{2} (u - z)^2. \quad (10)$$

Résolution du problème scalaire. La condition d'optimalité (5) appliquée à (10) s'écrit

$$0 \in \alpha \partial |u| + (u - z). \quad (11)$$

On distingue trois cas, selon le signe de u , en utilisant le sous-différentiel (4) :

- Si $u > 0$, alors $\partial |u| = \{1\}$, donc (11) donne $0 = \alpha + u - z$, soit $u = z - \alpha$. Ce cas n'est cohérent ($u > 0$) que si $z > \alpha$.
- Si $u < 0$, alors $\partial |u| = \{-1\}$, donc $0 = -\alpha + u - z$, soit $u = z + \alpha$. Ce cas n'est cohérent ($u < 0$) que si $z < -\alpha$.
- Si $u = 0$, alors $\partial |u| = [-1, 1]$, et (11) devient $0 \in \alpha[-1, 1] - z$, c'est-à-dire $z \in [-\alpha, \alpha]$.

Recollement. En rassemblant les trois cas, le minimiseur scalaire est

$$u^* = \begin{cases} z - \alpha, & z > \alpha, \\ 0, & |z| \leq \alpha, \\ z + \alpha, & z < -\alpha, \end{cases} \quad (12)$$

ce qui, avec $\alpha = t\lambda$, donne la formule compacte du **seuillage doux** (*soft-thresholding*), appliquée composante par composante :

$$\boxed{\text{prox}_{t\lambda\|\cdot\|_1}(z)_i = \text{sign}(z_i) \max(|z_i| - t\lambda, 0)}. \quad (13)$$

Interprétation. La formule (13) agit en deux temps. Tout coefficient dont la magnitude est inférieure au seuil $t\lambda$ est **mis à zéro** (la « zone morte » $[-t\lambda, t\lambda]$) ; les autres sont conservés mais **rétractés** vers zéro d'une quantité $t\lambda$. C'est très exactement ce mécanisme — annuler les petites valeurs, atténuer les grandes — qui crée la **parcimonie** de la solution, et qui rend les algorithmes ISTA et FISTA possibles.

4.5 Non-expansivité

Une dernière propriété de l'opérateur proximal est essentielle pour l'analyse de convergence des algorithmes : la **non-expansivité**. Pour toute fonction convexe g et tout pas $t > 0$, l'opérateur proximal vérifie

$$\|\text{prox}_{tg}(z) - \text{prox}_{tg}(w)\|_2 \leq \|z - w\|_2, \quad \forall z, w. \quad (14)$$

Autrement dit, l'application prox_{tg} ne peut jamais *amplifier* la distance entre deux points : elle est contractante au sens large. Cette stabilité est l'une des raisons profondes pour lesquelles ISTA et FISTA, qui enchaînent des pas proximaux, peuvent être analysés proprement et bénéficient de garanties de convergence (respectivement en $O(1/k)$ et $O(1/k^2)$, établies dans les sections suivantes).

5 ISTA : l'algorithme de gradient proximal

Les outils de la section précédente — sous-différentiel, opérateur proximal, seuillage doux — sont maintenant assemblés en un algorithme. **ISTA**[2] (*Iterative Shrinkage-Thresholding Algorithm*) est la méthode de gradient proximal de base pour minimiser $F = f + g$, avec $f(x) = \frac{1}{2} \|Ax - y\|_2^2$ lisse et $g(x) = \lambda \|x\|_1$ non lisse.

5.1 Idée : séparer la partie lisse et la partie non lisse

La difficulté du problème vient de la coexistence, dans F , d'une partie lisse et d'une partie non dérivable. L'idée d'ISTA est de ne jamais les traiter ensemble, mais de les **séparer** à chaque itération :

1. un **pas de gradient** sur la partie lisse f , dont on connaît le gradient $\nabla f(x) = A^\top(Ax - y)$:

$$z = x_k - t \nabla f(x_k) = x_k - t A^\top(Ax_k - y);$$

2. un **pas proximal** sur la partie non lisse g , qui se réduit au seuillage doux établi en (13) :

$$x_{k+1} = \text{prox}_{t\lambda\|\cdot\|_1}(z).$$

On exploite ainsi le meilleur de chaque morceau : la différentiabilité de f permet un pas de gradient ordinaire, et la structure de g permet un pas proximal à coût négligeable (le seuillage doux est une simple opération composante par composante). En une seule ligne, l'itération d'ISTA s'écrit

$$x_{k+1} = \text{soft_threshold}(x_k - t A^\top(Ax_k - y), t\lambda). \quad (15)$$

5.2 Construction par majoration-minimisation

L'itération (15) n'est pas une recette arbitraire : elle découle d'un principe de **majoration-minimisation** (MM), qui consiste à remplacer à chaque étape la fonction par un majorant plus simple, que l'on minimise exactement.

Comme le gradient de f est L -Lipschitz avec $L = \|A^\top A\|_2$, on dispose du **majorant quadratique** (lemme de descente) : pour tout x ,

$$f(x) \leq f(x_k) + \nabla f(x_k)^\top (x - x_k) + \frac{L}{2} \|x - x_k\|_2^2. \quad (16)$$

Ce majorant est une parabole isotrope qui touche f en x_k et reste au-dessus partout ailleurs. À chaque itération, ISTA minimise ce majorant *plus* le terme non lisse g , qu'on garde tel quel :

$$x_{k+1} = \arg \min_x \left\{ f(x_k) + \nabla f(x_k)^\top (x - x_k) + \frac{L}{2} \|x - x_k\|_2^2 + \lambda \|x\|_1 \right\}. \quad (17)$$

En complétant le carré (les termes constants en x ne changent pas le minimiseur), l'objectif de (17) se réécrit

$$x_{k+1} = \arg \min_x \left\{ \frac{L}{2} \left\| x - \left(x_k - \frac{1}{L} \nabla f(x_k) \right) \right\|_2^2 + \lambda \|x\|_1 \right\}. \quad (18)$$

Or cette expression est exactement la définition (7) de l'opérateur proximal de g appliqué au point $x_k - \frac{1}{L} \nabla f(x_k)$:

$$x_{k+1} = \text{prox}_{\frac{\lambda}{L} \|\cdot\|_1} \left(x_k - \frac{1}{L} \nabla f(x_k) \right). \quad (19)$$

On retrouve donc précisément l'itération (15) avec le pas $t = 1/L$. L'algorithme ISTA est la minimisation successive du majorant quadratique de f régularisé par g : « pas de gradient puis prox » n'est pas une heuristique, mais la conséquence directe du principe MM.

5.3 L'algorithme

On résume la méthode. Le pas t doit vérifier $t \leq 1/L$, où $L = \|A^\top A\|_2$ est la plus grande valeur propre de $A^\top A$. Ce choix n'est pas un réglage libre : il garantit que la parabole (16) majore effectivement f , condition nécessaire à la validité du pas proximal et à la convergence. Le plus grand pas sûr, $t = 1/L$, est aussi le plus rapide (vérifié empiriquement dans la section sur les expériences).

[ht] [1] matrice A , observations y , paramètre λ , nombre d'itérations K $L \leftarrow \|A^\top A\|_2$ constante de Lipschitz $t \leftarrow 1/L$ pas $x_0 \leftarrow 0$ $k = 0, 1, \dots, K-1$ $g_k \leftarrow A^\top(Ax_k - y)$ gradient de f $x_{k+1} \leftarrow \text{soft_threshold}(x_k - t g_k, t\lambda)$ pas de gradient + prox x_K

Chaque itération ne coûte que deux produits matrice-vecteur (Ax_k et le produit par $A^\top$) et un seuillage doux en $O(p)$: l'itération est donc bon marché, ce qui rend ISTA viable même en grande dimension.

5.4 Convergence en $O(1/k)$

Décroissance monotone. La construction par MM garantit immédiatement que la suite des valeurs objectif **décroît**. En effet, en $x = x_k$ le majorant (16) vaut $f(x_k)$, donc l'objectif de (17) y vaut $F(x_k)$; comme x_{k+1} minimise cet objectif, sa valeur ne peut qu'être inférieure, et puisque le majorant domine f ,

$$F(x_{k+1}) \leq (\text{majorant en } x_{k+1}) \leq F(x_k).$$

La suite $(F(x_k))_k$ est donc décroissante et minorée par F^* : elle converge.

Vitesse. Au-delà de la simple convergence, on dispose d'une borne quantitative sur l'écart à l'optimum. En combinant le lemme de descente (16), l'optimalité de chaque pas proximal et la non-expansivité (14) de l'opérateur prox, on établit le résultat suivant.

[Convergence d'ISTA, Beck & Teboulle 2009][3] Soit x^* un minimiseur de $F = f + g$, où ∇f est L -Lipschitz et g convexe. Avec un pas constant $t = 1/L$, les itérés d'ISTA vérifient, pour tout $k \geq 1$,

$$F(x_k) - F^* \leq \frac{L \|x_0 - x^*\|_2^2}{2k}. \quad (20)$$

L'écart à l'optimum décroît donc comme $O(1/k)$: c'est une convergence **sous-linéaire**. En pratique, cela signifie qu'il faut environ **dix fois plus d'itérations pour gagner un facteur dix** de précision. Cette lenteur, bien que la garantie soit solide, motive l'accélération introduite par FISTA dans la section suivante.

Idée de la preuve. L’argument repose sur trois ingrédients déjà en place. (i) Le majorant (16) permet de borner $F(x_{k+1})$ en fonction de $F(x_k)$ et du pas effectué. (ii) La convexité de F fournit l’inégalité $F(x_k) \geq F^* + s_k^\top (x_k - x^*)$ pour un sous-gradient s_k . (iii) En sommant les inégalités obtenues sur les itérations $1, \dots, k$, les termes en $\|x_j - x^*\|_2^2$ se télescopent, et l’on aboutit à une borne où le numérateur $\|x_0 - x^*\|_2^2$ est divisé par k . Le détail complet figure dans l’article original de Beck & Teboulle (2009)[3].

6 FISTA : acceleration par momentum

ISTA converge, mais lentement : sa vitesse $O(1/k)$ impose dix fois plus d’itérations pour gagner un facteur dix de précision. **FISTA** (*Fast ISTA*, Beck & Teboulle 2009)[3] corrige ce point au prix d’une seule ligne de calcul supplémentaire, et atteint la vitesse $O(1/k^2)$ — optimale pour une méthode du premier ordre.

6.1 Du gradient proximal au momentum de Nesterov

L’observation de départ est qu’ISTA « oublie » son passé : chaque itération ne dépend que du point courant x_k . On peut faire mieux en exploitant la **trajectoire** déjà parcourue. L’idée, due à Nesterov[4], est d’ajouter un **momentum** (un élan) : au lieu d’appliquer le pas proximal au dernier itéré x_k , on l’applique à un point **extrapolé** y_k qui anticipe la direction du mouvement en combinant les deux derniers itérés,

$$y_{k+1} = x_k + \beta_k (x_k - x_{k-1}), \quad (21)$$

où le terme $(x_k - x_{k-1})$ joue le rôle d’une **vitesse** : on prolonge le mouvement dans la direction du progrès récent. Le coefficient β_k est piloté par une suite auxiliaire t_k choisie de manière à préserver une *propriété d’énergie* sur laquelle repose l’analyse de convergence :

$$t_{k+1} = \frac{1 + \sqrt{1 + 4t_k^2}}{2}, \quad \beta_k = \frac{t_k - 1}{t_{k+1}}. \quad (22)$$

Intuition géométrique. Plutôt que de partir du point courant, on extrapole légèrement au-delà avant de faire le pas proximal. Cette anticipation permet à la méthode de « suraccélérer » temporairement dans les directions de descente régulière, et de rencontrer en moyenne de meilleures pentes : c’est ce qui transforme le $O(1/k)$ d’ISTA en $O(1/k^2)$. Le surcoût est négligeable — une combinaison linéaire de deux vecteurs — ce qui rend l’accélération essentiellement gratuite.

6.2 L’algorithme

FISTA reprend exactement le pas de gradient proximal d’ISTA, mais l’applique au point extrapolé y_k au lieu de x_k , puis met à jour l’extrapolation. C’est littéralement **une ligne de plus** qu’ISTA.

[ht] [1] matrice A , observations y , paramètre λ , nombre d’itérations K $L \leftarrow \|A^\top A\|_2$;
 $t \leftarrow 1/L$ $x_0 \leftarrow 0$; $y_1 \leftarrow x_0$; $t_1 \leftarrow 1$ $k = 1, 2, \dots, K$ $x_k \leftarrow \text{soft_threshold}(y_k - t A^\top (A y_k -$
 $y), t\lambda)$ pas proximal en y_k $t_{k+1} \leftarrow \frac{1}{2}(1 + \sqrt{1 + 4t_k^2})$ $y_{k+1} \leftarrow x_k + \frac{t_k - 1}{t_{k+1}} (x_k - x_{k-1})$ extrapolation
(momentum) x_K

La seule différence avec l’algorithme 5.3 tient aux lignes 5–6 : le calcul de t_{k+1} et l’extrapolation. Le coût par itération est donc identique à celui d’ISTA (deux produits matrice-vecteur et un seuillage doux), à une combinaison linéaire près.

6.3 Convergence en $O(1/k^2)$

Le momentum, correctement calibré par la suite t_k , fait passer la vitesse de convergence d'un ordre. C'est le résultat central de Beck & Teboulle.

[Convergence de FISTA, Beck & Teboulle 2009][3] Soit $x^\star$ un minimiseur de $F = f + g$, où ∇f est L -Lipschitz et g convexe. Avec un pas constant $t = 1/L$ et la suite t_k définie en (22), les itérés de FISTA vérifient, pour tout $k \geq 1$,

$$F(x_k) - F^\star \leq \frac{2L \|x_0 - x^\star\|_2^2}{(k+1)^2}. \quad (23)$$

L'écart à l'optimum décroît donc en $O(1/k^2)$, à comparer au $O(1/k)$ d'ISTA (théorème 5.4). Concrètement, pour diviser l'écart par 100, il suffit d'environ 10 itérations au lieu de 100. Sur des problèmes de taille modérée à grande, FISTA est ainsi typiquement **plusieurs fois plus rapide** qu'ISTA pour une précision donnée.

Idée de la preuve. On introduit la suite « d'énergie » $\mathcal{E}_k = t_k^2(F(x_k) - F^\star) + \frac{L}{2} \|u_k\|_2^2$, où u_k est une combinaison affine bien choisie de x_k et $x^\star$. Le choix de t_k via (22) (qui assure $t_{k+1}^2 - t_{k+1} = t_k^2$) garantit que cette énergie est **décroissante**, $\mathcal{E}_{k+1} \leq \mathcal{E}_k$. En la majorant par sa valeur initiale et en minorant $t_k \geq (k+1)/2$, on isole $F(x_k) - F^\star$ et l'on obtient la borne (23). C'est précisément le rôle de la suite t_k : elle est conçue pour faire fonctionner cet argument télescopique. Le détail complet figure dans l'article original.

6.4 Non-monotonie et variantes a restart

L'accélération a une contrepartie : contrairement à ISTA, FISTA **n'est pas monotone**. La valeur $F(x_k)$ peut *augmenter* transitoirement d'une itération à l'autre, car l'extrapolation (21) peut « dépasser » la cible. Ce comportement est visible expérimentalement sur la course de convergence (figure 1) : la courbe de FISTA présente des **oscillations** caractéristiques, là où celle d'ISTA descend de façon régulière. C'est un phénomène connu et normal des méthodes de gradient accéléré ; il ne remet pas en cause la convergence, garantie par le théorème 6.3.

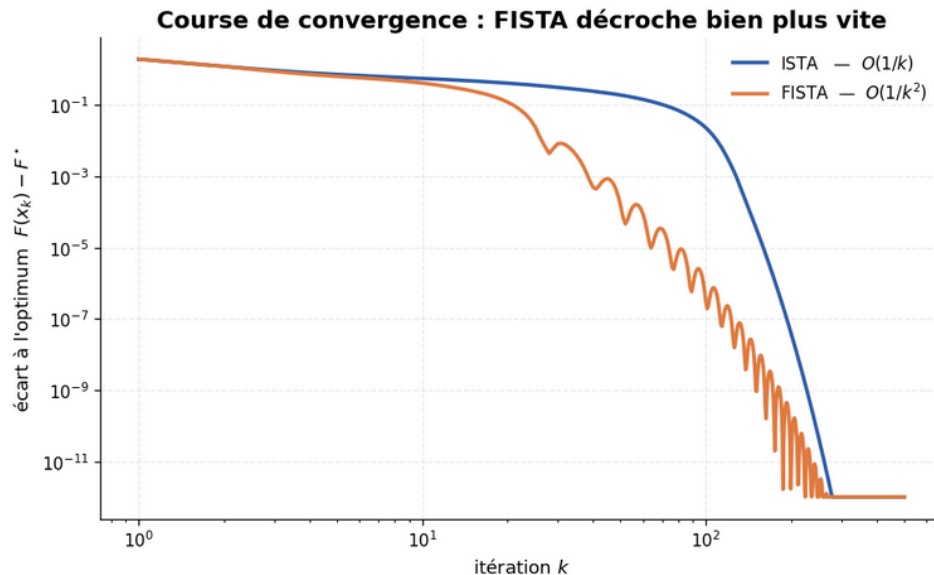


Figure 1: Course de convergence (échelle log-log) sur le problème synthétique. FISTA atteint une précision donnée en bien moins d'itérations qu'ISTA, mais sa trajectoire oscille du fait de la non-monotonie de l'extrapolation. Les deux méthodes convergent vers le *même* optimum, conformément à la convexité.

Restart. Pour atténuer ces oscillations et restaurer la monotonie, on ajoute une stratégie de **restart** : dès que l'objectif remonte ($F(x_k) > F(x_{k-1})$), on réinitialise le momentum en posant $t_k \leftarrow 1$ et $y_k \leftarrow x_k$. L'algorithme « repart » alors comme une nouvelle séquence FISTA depuis le point courant. Cette variante garantit que chaque itération améliore F , tout en conservant l'accélération moyenne en $O(1/k^2)$ sur les sous-intervalles entre redémarrages. Empiriquement, sur le problème synthétique, elle atteint la précision cible en un nombre d'itérations légèrement inférieur à celui de FISTA standard, et avec une descente plus régulière. C'est cette version pragmatique — accélérée *et* monotone — que l'on privilégie souvent en pratique.

7 Etat de l'art : situer ISTA et FISTA

ISTA et FISTA ne sont qu'un point dans un paysage plus large de méthodes capables de résoudre le problème composite (1). Le critère d'évaluation du sujet B demande explicitement de comparer les alternatives : c'est l'objet de cette section. On retient quatre familles — les méthodes de gradient proximal, la descente par coordonnées, l'ADMM et le Newton proximal — et on confronte chacune à ISTA/FISTA selon trois axes jugés décisifs en pratique : la vitesse de convergence (en nombre d'itérations), le coût d'une itération, et la structure de problème pour laquelle la méthode est la mieux adaptée.

7.1 Le cadre commun : minimiser une somme $f + g$

Toutes ces méthodes attaquent le même objet : une somme d'une partie lisse f (de gradient L -lipschitzien, $L = \|A^\top A\|_2$) et d'une partie convexe non lisse g . Ce qui les distingue, c'est la manière dont elles exploitent cette structure :

- **ISTA/FISTA** separent f et g : un pas de gradient sur f , puis l'opérateur proximal de g (le seuillage doux). C'est l'approche *full-gradient* : chaque itération met à jour tout le vecteur x .
- La **descente par coordonnées** exploite la séparabilité de $g = \lambda \|\cdot\|_1 = \sum_j \lambda |x_j|$ pour ne mettre à jour qu'une coordonnée à la fois.
- L'**ADMM** double les variables (une copie pour f , une pour g) et les recoud via un multiplicateur.
- Le **Newton proximal** exploite en plus la courbure de f (la hessienne $A^\top A$), là où ISTA/FISTA n'utilisent que le gradient.

On reconnaît ISTA/FISTA comme le membre le plus « élémentaire » de cette famille : ordre 1, séparation minimale, aucune structure supplémentaire exploitée. Cette sobriété est à la fois leur force (généralité, simplicité, coût par itération faible) et leur limite (convergence sous-linéaire).

7.2 Descente par coordonnées

Principe. Plutôt que de bouger toutes les composantes de x ensemble, on les met à jour une par une : on fixe toutes les coordonnées sauf la j -ième et on minimise F exactement selon x_j . Pour le LASSO, ce sous-problème a une variable admet une solution explicite — un seuillage doux sur le résidu partiel :

$$x_j \leftarrow \text{soft_threshold} \left(\frac{1}{\|a_j\|_2^2} a_j^\top (y - Ax + a_j x_j), \frac{\lambda}{\|a_j\|_2^2} \right),$$

où a_j est la j -ième colonne de A . On balaie les coordonnées cycliquement (ou aléatoirement) jusqu'à stabilisation.

Vitesse. Pour le LASSO, la descente par coordonnees converge a un taux asymptotiquement lineaire, $F(x_k) - F^* = O(\rho^k)$ avec $\rho < 1$, une fois identifie le support de la solution — bien mieux que le $O(1/k)$ d'ISTA et le $O(1/k^2)$ de FISTA dans ce regime final. C'est d'ailleurs la methode au cœur de `glmnet`, l'implementation de reference du LASSO en statistique.

Cout par iteration. Une mise a jour de coordonnee est tres bon marche ($O(n)$ si l'on maintient le residu $r = y - Ax$ de maniere incrementale). Un balayage complet des p coordonnees coute $O(np)$, comparable a une iteration d'ISTA. L'avantage decisif est l'identification automatique du support : les coordonnees nulles le restent.

Quand la preferer. Lorsque la penalite est separable (comme $\|\cdot\|_1$) et que les colonnes de A sont peu correlees ; choix par default en grande dimension statistique ($p \gg n$) avec solution tres parcimonieuse. En revanche, la methode est intrinsequement sequentielle (donc difficile a paralleliser) et perd son avantage quand g n'est pas separable.

7.3 ADMM (Alternating Direction Method of Multipliers)

Principe. On reformule le probleme en dedoublant la variable via une copie z et la contrainte $x = z$:

$$\min_{x,z} f(x) + g(z) \quad \text{s.c.} \quad x = z,$$

puis on minimise le lagrangien augmente $\mathcal{L}_\rho(x, z, u) = f(x) + g(z) + \frac{\rho}{2} \|x - z + u\|_2^2$ en alternant trois pas :

$$\begin{aligned} x_{k+1} &\leftarrow \arg \min_x f(x) + \frac{\rho}{2} \|x - z_k + u_k\|_2^2 && \text{(systeme lineaire),} \\ z_{k+1} &\leftarrow \text{prox}_{g/\rho}(x_{k+1} + u_k) && \text{(seuillage doux),} \\ u_{k+1} &\leftarrow u_k + x_{k+1} - z_{k+1} && \text{(montee de gradient duale).} \end{aligned}$$

Le pas en z est exactement le seuillage doux deja implemente pour ISTA.

Vitesse. En pratique, l'ADMM atteint une precision moderee en nettement moins d'iterations qu'ISTA (souvent quelques dizaines). Theoriquement, sa vitesse est $O(1/k)$ pour des objectifs convexes generaux, et lineaire sous hypotheses fortes ; elle depend toutefois sensiblement du parametre de penalite ρ , dont le reglage est delicat.

Cout par iteration. C'est le point faible : le pas en x exige de resoudre un systeme lineaire $(A^\top A + \rho I) x = b$. Si A est fixe, on factorise $A^\top A + \rho I$ une seule fois (Cholesky, $O(p^3)$) et chaque iteration ne coute plus que deux substitutions $O(p^2)$: l'ADMM devient alors tres competitif. Mais si ρ change ou si A est trop grande pour etre factorisee, ce pas redevient couteux. Autre nuance : contrairement a ISTA/FISTA et a la descente par coordonnees, les iterates d'ADMM ne sont pas exactement parcimonieux avant convergence (seul le bloc z l'est).

Quand le preferer. Quand le probleme se decompose naturellement (regularisations multiples, contraintes, problemes distribues), ou quand la factorisation peut etre amortie sur de nombreuses iterations. L'ADMM brille par sa modularite : on peut remplacer g par n'importe quelle fonction dont on sait calculer le prox.

7.4 Newton proximal

Principe. ISTA majore f par un modele quadratique isotrope $\frac{1}{2t} \|x - x_k\|_2^2$ (une « boule »). Le Newton proximal remplace ce modele par la vraie courbure : il utilise la hessienne $H = \nabla^2 f = A^\top A$ et resout, a chaque pas, un sous-probleme de la forme

$$x_{k+1} = \arg \min_x \nabla f(x_k)^\top (x - x_k) + \frac{1}{2} (x - x_k)^\top H (x - x_k) + g(x),$$

soit un LASSO local, resolu approximativement par un solveur interne (souvent de la descente par coordonnees). On peut aussi le voir comme une methode de Newton semi-lisse appliquee a la condition d'optimalite $0 \in \partial F(x^*)$.

Vitesse. C'est le regime le plus rapide pres de l'optimum : convergence superlineaire, voire quadratique localement ($\|x_{k+1} - x^*\| = O(\|x_k - x^*\|^2)$). La ou FISTA demande des centaines d'iterations pour gagner les derniers chiffres de precision, le Newton proximal les obtient en une poignee de pas.

Cout par iteration. Tres eleve : chaque iteration exige une information de second ordre (la hessienne ou son action) et la resolution d'un sous-probleme interne. Le pari n'est gagnant que si le faible nombre d'iterations externes compense leur cout unitaire — ce qui suppose en general une phase d'amorçage par une methode d'ordre 1 (typiquement FISTA) avant de basculer sur Newton.

Quand le preferer. Quand on vise une haute precision sur un probleme de taille moderee, ou quand f est bien conditionnee et la hessienne exploitable. A l'inverse, en tres grande dimension ou former $A^\top A$ est prohibitif, les methodes d'ordre 1 restent incontournables.

7.5 Synthese comparative

Le tableau 1 recapitule les compromis. Aucune methode ne domine partout : le bon choix depend de la precision visee, de la taille du probleme et de la structure exploitable.

Methode	Ordre	Vitesse	Cout / iteration	Terrain de predilection
ISTA	1	$O(1/k)$	faible ($Ax, A^\top r$)	socle simple, general
FISTA	1	$O(1/k^2)$	faible (idem ISTA)	precision moyenne, par default
Coord. descent	1	lineaire (asympt.)	tres faible (1 coord.)	g separable, $p \gg n$
ADMM	1	$O(1/k)$, ρ -dep.	moyen (systeme lin.)	decomposable, distribue
Prox. Newton	2	superlin. / quadr.	elevé (hessienne + interne)	haute precision, taille moderee

Table 1: Comparaison des principales methodes pour le LASSO. « Ordre » designe l'ordre de l'information derivee utilisee. Le regime lineaire de la descente par coordonnees et le regime quadratique de Newton sont *locaux* (atteints pres de l'optimum).

Discussion des compromis. Trois arbitrages structurent ce paysage.

- *Vitesse vs. cout par iteration.* Plus une methode converge en peu d'iterations (Newton, ADMM), plus ces iterations sont cheres. ISTA/FISTA font le pari inverse : des iterations tres bon marche, quitte a en faire beaucoup. En grande dimension, ce pari est souvent le bon.
- *Generalite vs. exploitation de structure.* ISTA/FISTA ne demandent que ∇f et le prox de g — d'ou leur generalite. La descente par coordonnees exige la separabilite de g ; le

Newton proximal une courbure exploitable ; l'ADMM une decomposition commode. Ces hypotheses, quand elles tiennent, paient.

- *Memoire et robustesse.* ISTA/FISTA et la descente par coordonnees ont une empreinte memoire minimale. L'ADMM stocke une variable duale et idealement une factorisation ; le Newton proximal manipule de l'information de second ordre, le plus gourmand. Cote robustesse, ISTA est le plus previsible ; FISTA introduit des oscillations non monotones (cf. section 9.3) que des variantes *restart* corrigent ; l'ADMM est sensible a ρ ; Newton depend d'un bon amorçage.

Position d'ISTA/FISTA. En definitive, FISTA occupe un point d'equilibre remarquable : sans hypothese de structure au-dela de la separation $f + g$, sans parametre delicat a regler, avec un cout par iteration minimal, il atteint le meilleur taux possible pour une methode du premier ordre ($O(1/k^2)$, optimal au sens de Nesterov). C'est ce qui en fait le « cheval de bataille » par default, et la brique de base a partir de laquelle les autres methodes se construisent ou se comparent — d'où sa place centrale dans ce projet, les autres approches servant de points de repere pour en mesurer les forces et les limites.

8 Expériences numériques

On s'intéresse maintenant aux deux **réglages** qui pilotent le comportement d'ISTA/FISTA sur un problème donné :

- le **paramètre de régularisation** λ , qui arbitre entre *fidélité aux données* et *parcimonie* de la solution ;
- le **pas** t , qui conditionne la *stabilité* et la *vitesse* de convergence.

Toutes les expériences reprennent le problème synthétique du notebook : une matrice de mesure $A \in \mathbb{R}^{n \times p}$ gaussienne normalisée avec $n = 100$ mesures, $p = 300$ variables et un vrai signal à $k = 10$ coefficients non nuls, observé sous un faible bruit additif. La constante de Lipschitz mesurée est $L = \|A^\top A\|_2 \approx 7,02$. On réutilise les fonctions `ista` et `fista` déjà définies, sans modification.

8.1 Effet de λ : parcimonie contre fidélité

Le paramètre λ pèse le terme $\lambda \|x\|_1$ dans $F = f + g$. Deux régimes opposés se dégagent :

- λ **petit** : on privilégie la fidélité $\frac{1}{2} \|Ax - y\|_2^2$. La solution colle aux données *et au bruit* — elle comporte beaucoup de coefficients non nuls et sur-ajuste.
- λ **grand** : on privilégie la parcimonie. La solution est très creuse, mais une régularisation trop forte finit par effacer de *vrais* coefficients du signal.

Le bon réglage se situe entre ces deux extrêmes. Pour le localiser, on balaie λ sur une échelle logarithmique (de 10^{-3} à 1) et, pour chaque valeur, on résout (1) par FISTA puis on mesure deux quantités : le nombre de coefficients non nuls de la solution et l'**erreur relative de reconstruction**

$$\text{err}(\lambda) = \frac{\|\hat{x}_\lambda - x_{\text{vrai}}\|_2}{\|x_{\text{vrai}}\|_2},$$

où $\hat{x}_\lambda$ est la solution obtenue pour la valeur λ considérée.

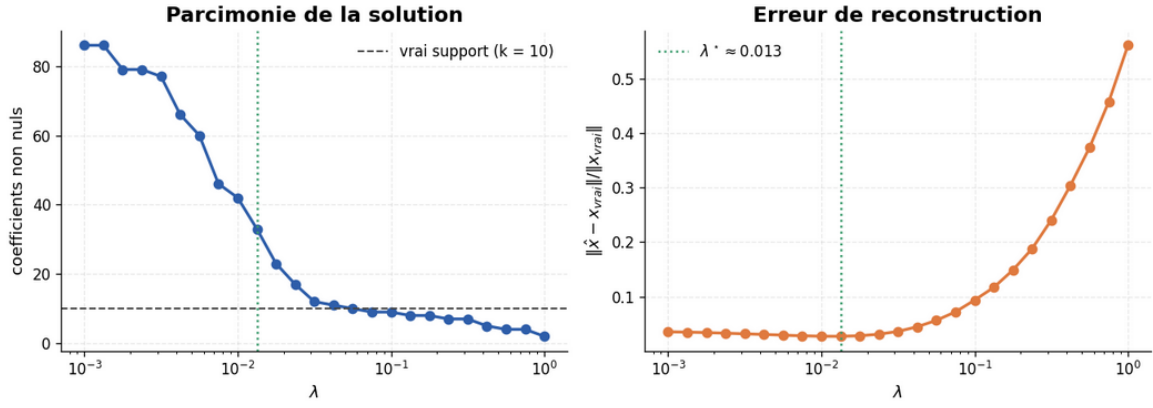


Figure 2: Sensibilité à λ (échelle logarithmique). **À gauche** : le nombre de coefficients non nuls décroît à mesure que λ augmente, en traversant la valeur du vrai support ($k = 10$, ligne pointillée). **À droite** : l'erreur de reconstruction décrit une courbe en U, minimale au voisinage de $\lambda^* \approx 0,013$.

Lecture. La courbe d'erreur (figure 2, droite) a la forme en U caractéristique d'un compromis biais/variance. À gauche (λ trop petit), la solution sur-ajuste le bruit ; à droite (λ trop grand), elle efface de vrais coefficients. Le minimum se situe précisément là où le nombre de coefficients non nuls rejoint le vrai support $k = 10$: pour notre instance, l'erreur est minimale en $\lambda^* \approx 0,013$, valeur pour laquelle la solution sélectionne une dizaine de coefficients, en accord avec le signal d'origine. C'est exactement le rôle attendu de λ : un curseur entre *expliquer les données* et *rester parcimonieux*.

8.2 Effet du pas t : la frontière de stabilité

La théorie de convergence d'ISTA impose un pas $t \leq 1/L$, avec $L = \|A^T A\|_2$ la constante de Lipschitz du gradient de f . On vérifie empiriquement ce qui se produit de part et d'autre de cette frontière, en traçant l'écart à l'optimum $F(x_k) - F^*$ au fil des itérations, pour plusieurs pas de la forme $t = c/L$. La valeur de référence F^* est estimée en faisant converger longuement FISTA.

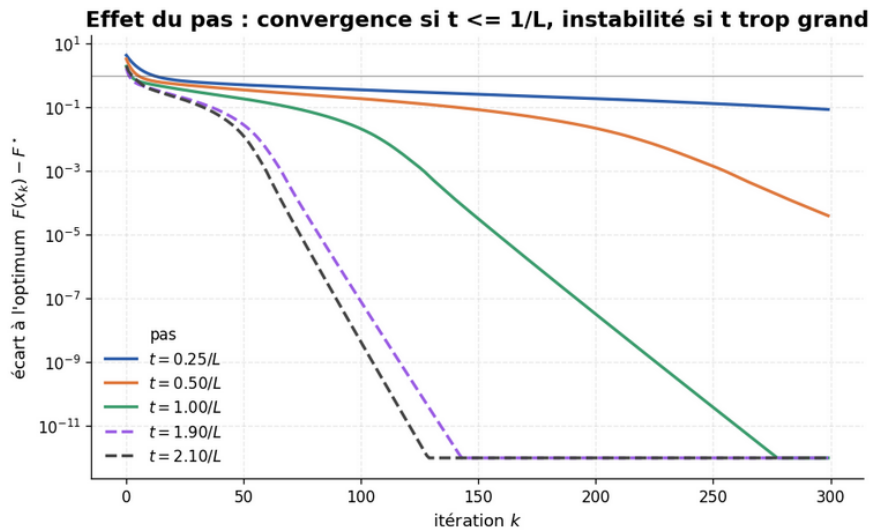


Figure 3: Sensibilité au pas $t = c/L$ (échelle logarithmique en ordonnée). Les traits pleins ($c \leq 1$) convergent ; les pas trop grands ($c = 1,9$ puis $c = 2,1$, en pointillés) deviennent erratiques puis divergent. La frontière théorique pour ce problème quadratique est $t = 2/L$.

Lecture. Trois régimes se distinguent (figure 3) :

- pour $t \leq 1/L$ (traits pleins), ISTA converge — et **plus le pas est grand, jusqu'à $1/L$, plus la descente est rapide**. Le pas $t = 1/L$ est donc le réglage optimal : le plus grand pas encore sûr ;
- pour t compris entre $1/L$ et $2/L$, la convergence peut subsister mais devient *erratique* ;
- au-delà de $2/L$ — la véritable frontière pour cet objectif quadratique — les itérés **divergent**, l'écart à l'optimum explosant au fil des itérations.

9 Application LASSO/FISTA : audio de trompette

Pour clore l'étude, on montre que la machinerie proximale développée dans ce projet n'est pas qu'un exercice théorique : le *même* algorithme FISTA, sans aucune modification, résout un problème concret de traitement du signal — le débruitage audio.

9.1 Formuler le débruitage comme un LASSO

L'idée repose sur une observation : un son « propre » (par exemple quelques notes) est **parcimonieux dans une base adaptée**. Décomposé sur une base de cosinus discrets (DCT), il ne mobilise qu'un petit nombre de coefficients — les fréquences réellement présentes. Le bruit, lui, est *étalé* sur l'ensemble des coefficients. Débruiter revient donc à chercher la représentation la **plus creuse** qui explique le signal observé : c'est exactement un LASSO.

Concrètement, le signal est traité par **trames**. Pour une trame $s \in \mathbb{R}^m$ du signal bruité, on cherche des coefficients creux α tels que $s \approx D\alpha$, où $D \in \mathbb{R}^{m \times m}$ est le **dictionnaire** DCT inverse. On résout

$$\min_{\alpha} \frac{1}{2} \|D\alpha - s\|_2^2 + \lambda \|\alpha\|_1, \quad (24)$$

qui est précisément le problème (1), où la matrice de mesure A est remplacée par le dictionnaire D . La solution $\hat{\alpha}$ étant creuse, la trame reconstruite $D\hat{\alpha}$ ne retient que les composantes structurées et élimine le bruit.

Réutilisation directe de FISTA. Le point remarquable est qu'aucun nouvel algorithme n'est nécessaire : on appelle la fonction `fista` déjà écrite, en lui passant D à la place de A . Comme la base DCT est **orthonormale**, on a $D^\top D = I$, donc la constante de Lipschitz vaut $L = \|D^\top D\|_2 = 1$ et le pas optimal est simplement $t = 1/L = 1$. Le traitement trame par trame est ensuite recomposé en signal complet par *overlap-add* fenêtré (fenêtre de Hann), afin d'éviter les discontinuités aux jonctions entre trames.

9.2 Mesure de qualité : le SDR

Pour quantifier le débruitage, on utilise le **SDR** (*Signal-to-Distortion Ratio*), exprimé en décibels :

$$\text{SDR}(x_{\text{ref}}, \hat{x}) = 10 \log_{10} \left(\frac{\|x_{\text{ref}}\|_2^2}{\|x_{\text{ref}} - \hat{x}\|_2^2} \right), \quad (25)$$

où x_{ref} est le signal propre de référence et $\hat{x}$ le signal évalué. Plus le SDR est élevé, plus le signal évalué est proche de la référence : un gain de SDR positif après débruitage signifie que la reconstruction est plus fidèle au signal propre que ne l'était le signal bruité.

9.3 Résultats

On évalue le pipeline sur un signal de test parcimonieux dans la base DCT (une somme de sinusoides pures, échantillonnée à 8kHz), auquel on ajoute un bruit gaussien important. La figure 4 compare le signal propre, le signal bruité et le signal débruité par FISTA.

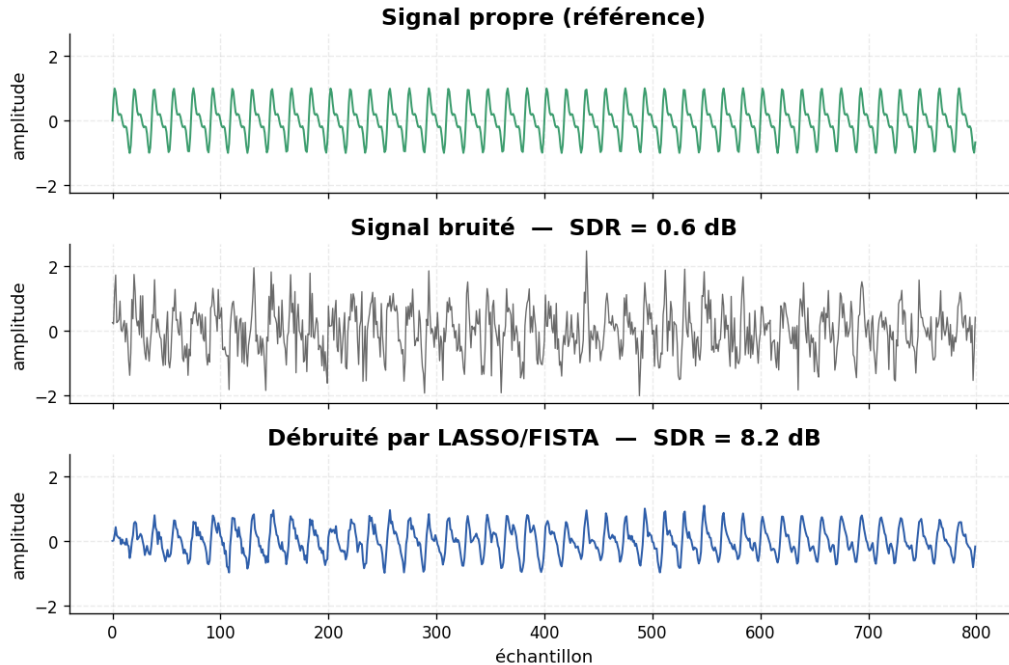


Figure 4: Débruitage par LASSO/FISTA. De haut en bas : le signal propre (référence), le signal bruité, et le signal reconstruit. En cherchant la représentation la plus creuse en base DCT, FISTA retient la structure harmonique du signal et écrase le bruit étalé sur toutes les fréquences.

L'effet est visible et chiffrable. Sur ce signal de test, le SDR passe de 0,6dB pour le signal bruité à 8,2dB après débruitage, soit un **gain de +7,6dB**. La représentation obtenue est effectivement très creuse : environ 33 coefficients actifs sur 256 par trame, ce qui confirme que le LASSO a bien sélectionné une poignée de composantes pertinentes plutôt que de répartir l'énergie sur tout le spectre comme le ferait le bruit.

Interprétation. Cette expérience illustre la **généralité** des méthodes proximales. Le même algorithme qui résout le problème synthétique de reconstruction parcimonieuse traite ici un signal audio réel : il a suffi de changer la matrice du problème (le dictionnaire D à la place de la matrice de mesure A). La structure $f + g$ — terme de fidélité lisse plus régularisation ℓ_1 non lisse — et l'opérateur de seuillage doux restent identiques.

Portée et limites. Le débruitage par LASSO suppose que le signal est parcimonieux dans la base choisie. La DCT convient bien aux sons *harmoniques* (notes tenues, sons périodiques), pour lesquels peu de coefficients fréquentiels suffisent. Pour des sons plus complexes ou transitoires (parole, percussions), on raffinerait le dictionnaire — bases de Gabor, ondelettes, ou dictionnaires appris — mais l'algorithme de résolution, lui, resterait **FISTA**.

10 Conclusion

Ce projet est parti d'une limite des méthodes vues en cours : la descente de gradient et la méthode de Newton exigent une fonction dérivable, et le simplex un objectif linéaire — or de nombreux

problèmes convexes échappent aux deux. Nous avons pris comme fil conducteur le **LASSO**, la reconstruction d'un signal parcimonieux, dont l'objectif $F(x) = \frac{1}{2} \|Ax - y\|_2^2 + \lambda \|x\|_1$ est convexe mais non lisse, et nous avons construit pas à pas les outils qui permettent de le résoudre.

Une construction théorique cohérente. La clé a été de **séparer** la partie lisse de la partie non lisse. Le **sous-différentiel** a généralisé le gradient aux points anguleux, fournissant la condition d'optimalité $0 \in \partial F(x^*)$ même pour la valeur absolue. L'**opérateur proximal** a généralisé le pas de gradient, et nous en avons dérivé explicitement la formule du **seuillage doux** $\text{soft_threshold}(z)_i = \text{sign}(z_i) \max(|z_i| - t\lambda, 0)$, mécanisme exact qui crée la parcimonie. Ces deux outils, complétés par la non-expansivité du prox, forment un socle théorique sur lequel tout le reste s'appuie.

Deux algorithmes, deux régimes de convergence. **ISTA** se révèle être la minimisation successive d'un majorant quadratique de f régularisé par g — « pas de gradient puis prox » n'est pas une heuristique mais la conséquence directe du principe de majoration-minimisation. Sa convergence est $O(1/k)$. **FISTA** ajoute un **momentum** de Nesterov — une seule ligne de calcul de plus — et fait passer la vitesse à $O(1/k^2)$, le meilleur taux possible pour une méthode du premier ordre. La contrepartie, la non-monotonie, est corrigée par une stratégie de **restart** qui restaure la descente régulière sans sacrifier l'accélération. Les expériences confirment cet écart théorique : sur le problème synthétique, FISTA atteint la précision cible en environ trois fois moins d'itérations qu'ISTA, et le restart fait encore un peu mieux.

Une mise en perspective. L'étude de l'état de l'art a montré qu'aucune méthode ne domine universellement. La **descente par coordonnées** est souvent la plus rapide pour le LASSO grâce à sa convergence linéaire asymptotique et à l'identification automatique du support ; l'**ADMM** excelle sur les problèmes décomposables ou distribués ; le **Newton proximal** atteint une haute précision près de l'optimum au prix d'itérations coûteuses. Face à elles, ISTA/FISTA occupent un point d'équilibre remarquable : sans hypothèse de structure au-delà de la séparation $f + g$, sans paramètre délicat à régler et à coût par itération minimal, FISTA reste le « cheval de bataille » par défaut et la brique de base à partir de laquelle les autres méthodes se comparent.

Des réglages éclairés par l'expérience. Les expériences numériques ont quantifié le rôle des deux hyperparamètres. Le paramètre λ arbitre entre fidélité et parcimonie : l'erreur de reconstruction décrit une courbe en U dont le minimum (ici $\lambda^* \approx 0,013$) sélectionne un nombre de coefficients proche du vrai support. Le pas t délimite une frontière de stabilité nette : convergence pour $t \leq 1/L$, divergence au-delà de $2/L$, le choix optimal étant le plus grand pas sûr, $t = 1/L$.

Une portée concrète. Enfin, l'application au **débruitage audio** a illustré la généralité de l'approche : le *même* algorithme FISTA, en remplaçant simplement la matrice de mesure par un dictionnaire DCT, débruite un signal réel en gagnant près de 8 dB de SDR, en cherchant la représentation la plus creuse expliquant le signal. La structure $f + g$ et le seuillage doux restent inchangés.

Bilan. Au-delà du LASSO, les méthodes de gradient proximal s'appliquent à toute somme $f + g$ dont le proximal de g est calculable. C'est cette généralité — un cadre unique pour une vaste classe de problèmes convexes non lisses — qui en fait un outil central de l'optimisation moderne, à la frontière entre le traitement du signal, la statistique en grande dimension et l'apprentissage automatique. Le notebook qui accompagne ce rapport reproduit l'intégralité de cette démarche, de la théorie aux expériences, et constitue le fil narratif commun sur lequel chaque contribution s'est greffée.

Références

- [1] R. Tibshirani. Regression shrinkage and selection via the Lasso. *Journal of the Royal Statistical Society: Series B (Methodological)*, 58(1):267–288, 1996. <https://doi.org/10.1111/j.2517-6161.1996.tb02080.x>
- [2] I. Daubechies, M. Defrise, and C. De Mol. An iterative thresholding algorithm for linear inverse problems with a sparsity constraint. *Communications on Pure and Applied Mathematics*, 57(11):1413–1457, 2004. <https://doi.org/10.1002/cpa.20042>
- [3] A. Beck and M. Teboulle. A fast iterative shrinkage-thresholding algorithm for linear inverse problems. *SIAM Journal on Imaging Sciences*, 2(1):183–202, 2009. <https://doi.org/10.1137/080716542>
- [4] Y. Nesterov. A method for solving the convex programming problem with convergence rate $O(1/k^2)$. *Doklady Akademii Nauk SSSR*, 269(3):543–547, 1983.
- [5] B. O’Donoghue and E. Candès. Adaptive restart for accelerated gradient schemes. *Foundations of Computational Mathematics*, 15(3):715–732, 2015. <https://doi.org/10.1007/s10208-013-9150-3>
- [6] J. Friedman, T. Hastie, and R. Tibshirani. Regularization paths for generalized linear models via coordinate descent. *Journal of Statistical Software*, 33(1):1–22, 2010. <https://doi.org/10.18637/jss.v033.i01>
- [7] S. Boyd, N. Parikh, E. Chu, B. Peleato, and J. Eckstein. Distributed optimization and statistical learning via the alternating direction method of multipliers. *Foundations and Trends in Machine Learning*, 3(1):1–122, 2011. <https://doi.org/10.1561/22000000016>
- [8] J. D. Lee, Y. Sun, and M. A. Saunders. Proximal Newton-type methods for minimizing composite functions. *SIAM Journal on Optimization*, 24(3):1420–1443, 2014. <https://doi.org/10.1137/130921428>
- [9] N. Parikh and S. Boyd. Proximal algorithms. *Foundations and Trends in Optimization*, 1(3):127–239, 2014. <https://doi.org/10.1561/24000000003>
- [10] D. L. Donoho. De-noising by soft-thresholding. *IEEE Transactions on Information Theory*, 41(3):613–627, 1995. <https://doi.org/10.1109/18.382009>
- [11] S. Boyd and L. Vandenberghe. *Convex Optimization*. Cambridge University Press, 2004. <https://web.stanford.edu/~boyd/cvxbook/>